Backend API Requirements & Issues

🔴 Critical Issues Found

1. Brand ID Type Mismatch

Problem: Your API has inconsistent ID types:

- `/api/brands` returns `uuid` (string)
- `/api/media_contents/brand/{brandId}` expects `int64` (number)

Solution Options:

Option A: Use UUID for brandId everywhere (Recommended)

```csharp
// Change MediaContents endpoint
[HttpGet("brand/{brandId:guid}")]
public async Task<ActionResult<List<MediaContentDto>>> GetByBrand(Guid brandId)
```

Option B: Add numeric ID to Brand model

```csharp
public class BrandDto
{
    public Guid Uuid { get; set; }
    public long Id { get; set; }  // Add this
    public string Name { get; set; }
    public string Icon { get; set; }
}
```

2. Missing Query Parameter for Schedules

Problem: No way to filter schedules by brandId

```
GET /api/schedules ← Returns ALL schedules
```

Required Addition:

```csharp
[HttpGet]
public async Task<ActionResult<List<ScheduleDto>>> GetSchedules(
    [FromQuery] Guid? brandId = null)
{
    var query = _context.Schedules.AsQueryable();

    if (brandId.HasValue)
    {
        query = query.Where(s => s.BrandId == brandId.Value);
    }

    return await query.ToListAsync();
}
```

3. Missing Response DTOs

Problem: Some endpoints return `void` but frontend needs the created object

Required Changes:

```csharp
```

```csharp
// Schedule endpoints should return ScheduleDto
[HttpPost]
public async Task<ActionResult<ScheduleDto>> CreateSchedule(ScheduleCreateDto dto)
{
    var schedule = // ... create schedule
    await _context.SaveChangesAsync();
    return Ok(_mapper.Map<ScheduleDto>(schedule));  // Return created object
}

[HttpPut("{id:guid}")]
public async Task<ActionResult<ScheduleDto>> UpdateSchedule(Guid id, ScheduleUpdateDto dto)
{
    var schedule = // ... update schedule
    await _context.SaveChangesAsync();
    return Ok(_mapper.Map<ScheduleDto>(schedule));  // Return updated object
}
```

📋 Missing DTO Definitions

Based on your OpenAPI spec, you need these DTOs:

ScheduleDto

```csharp
```

```csharp
public class ScheduleDto
{
    public Guid Id { get; set; }
    public DateTime ScheduledAt { get; set; }
    public List<string> Platforms { get; set; }
    public string Status { get; set; }  // "success", "failed", "scheduled"
    public string MediaType { get; set; }  // "image", "video"
    public string Title { get; set; }
    public string? Description { get; set; }
    public Guid? BrandId { get; set; }
    public Guid? MediaContentId { get; set; }
    public long? TenantId { get; set; }
}
```

ScheduleCreateDto

```csharp
```

```csharp
public class ScheduleCreateDto
{
    [Required]
    public DateTime ScheduledAt { get; set; }

    [Required]
    public List<string> Platforms { get; set; }

    [Required]
    public string Status { get; set; }

    [Required]
    public string MediaType { get; set; }

    [Required]
    public string Title { get; set; }

    public string? Description { get; set; }
    public Guid? BrandId { get; set; }
    public Guid? MediaContentId { get; set; }
}
```

ScheduleUpdateDto

csharp

```
public class ScheduleUpdateDto
{
    public DateTime? ScheduledAt { get; set; }
    public List<string>? Platforms { get; set; }
    public string? Status { get; set; }
    public string? MediaType { get; set; }
    public string? Title { get; set; }
    public string? Description { get; set; }
}
```

MediaContentDto

```csharp
public class MediaContentDto
{
    public Guid Uuid { get; set; }
    public string Title { get; set; }
    public string Thumbnail { get; set; }
    public bool Favorite { get; set; }
    public string Origin { get; set; }  // "presence", "external"
    public long? BrandId { get; set; }
    public long? ListId { get; set; }
    public string? ContentType { get; set; }
    public string? Url { get; set; }
}
```

🔧 Required Backend Changes Summary

1. Fix Brand ID Consistency

```csharp
```

```
// Change this endpoint
[HttpGet("brand/{brandId:int}")]  // Current
[HttpGet("brand/{brandId:guid}")]  // Change to this

// Or add numeric ID to BrandDto
```

2. Add Query Parameter to Schedules

```csharp
[HttpGet]
public async Task<ActionResult<List<ScheduleDto>>> GetSchedules(
    [FromQuery] Guid? brandId = null,
    [FromQuery] DateTime? from = null,
    [FromQuery] DateTime? to = null)
{
    var query = _context.Schedules.AsQueryable();

    if (brandId.HasValue)
        query = query.Where(s => s.BrandId == brandId.Value);

    if (from.HasValue)
        query = query.Where(s => s.ScheduledAt >= from.Value);

    if (to.HasValue)
        query = query.Where(s => s.ScheduledAt <= to.Value);

    return await query.ToListAsync();
}
```

3. Return Created/Updated Objects

```csharp
// Instead of returning void, return the object
[HttpPost]
public async Task<ActionResult<ScheduleDto>> CreateSchedule(...)
{
    // ... create logic
    return Ok(scheduleDto);  // Return created object
}
```

4. Add CORS Configuration

```csharp
// In Program.cs or Startup.cs
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:5173")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});

// Use CORS
app.UseCors("AllowFrontend");
```

5. Add Firebase Authentication

```csharp
```

```
// Install package
// dotnet add package FirebaseAdmin

// Configure
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "https://securetoken.google.com/your-project-id";
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = "https://securetoken.google.com/your-project-id",
            ValidateAudience = true,
            ValidAudience = "your-project-id",
            ValidateLifetime = true
        };
    });

// Use authentication
app.UseAuthentication();
app.UseAuthorization();

// Protect endpoints
[Authorize]
[ApiController]
public class BrandsController : ControllerBase
{
    // Your endpoints
}
```

📝 Frontend Hook Mapping

Based on your actual API, here's the correct mapping:

Frontend Concept	Backend Endpoint	Hook to Use
Brands	`/api/brands`	`useBrands`
Posts/Schedules	`/api/schedules`	`useSchedules`
Content	`/api/media_contents/brand/{brandId}`	`useMediaContents`
Content Lists	`/api/media_contents/list/{listId}`	`useContentLists` (new)

🆕 Additional Hooks Needed

useContentLists

For managing content lists (My Videos, My Images, etc.)

```typescript
// Frontend needs this hook
export const useContentLists = (brandId: string) => {
  // Fetch lists for a brand
  // Create custom lists
  // Update list metadata
  // Delete lists
};
```

Required Backend Endpoints:

```csharp
```

```
[HttpGet("lists/brand/{brandId:guid}")]
public async Task<ActionResult<List<MediaContentListDto>>> GetListsByBrand(Guid brandId)

[HttpGet("lists/{listId:int}")]
public async Task<ActionResult<MediaContentListDto>> GetList(int listId)

[HttpPut("lists/{listId:int}")]
public async Task<ActionResult> UpdateList(int listId, MediaContentListUpdateDto dto)

[HttpDelete("lists/{listId:int}")]
public async Task<ActionResult> DeleteList(int listId)
```

useTenants

For multi-tenancy support (if needed)

```typescript
export const useTenants = () => {
  // Fetch tenants
  // Switch tenant context
};
```

🐛 Fixing the 404 Error

The 404 error occurs because:

1. **Wrong endpoint path** - Frontend called `/api/brands` but proxy failed
2. **CORS not configured** - Backend rejected the request
3. **Authentication required** - Backend requires auth token

Debug Steps:

1. **Test backend directly:**

```bash
curl http://192.168.1.12:5112/api/brands
```

2. **Check CORS:**

```bash
curl -H "Origin: http://localhost:5173" \
    -H "Access-Control-Request-Method: GET" \
    -X OPTIONS \
    http://192.168.1.12:5112/api/brands
```

3. **Test with auth:**

```bash
curl -H "Authorization: Bearer YOUR_FIREBASE_TOKEN" \
    http://192.168.1.12:5112/api/brands
```

📦 Required Package Installations

Backend

```bash
dotnet add package FirebaseAdmin
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package Swashbuckle.AspNetCore  # If not already added
```

Frontend

```bash
npm install --save-dev @types/node
```

✅ Checklist

- [ ] Fix brandId type consistency (use Guid everywhere)
- [ ] Add query parameter support to `/api/schedules`
- [ ] Return DTOs from POST/PUT endpoints (not void)
- [ ] Add CORS configuration
- [ ] Add Firebase authentication middleware
- [ ] Add DTO definitions for Schedule endpoints
- [ ] Add endpoint to fetch lists by brand
- [ ] Test all endpoints with Postman/curl
- [ ] Update OpenAPI spec after changes

🔄 Data Flow

```
Frontend                Backend
--------                -------
useSchedules    →  GET /api/schedules?brandId={uuid}
            ←  ScheduleDto[]

useMediaContents →  GET /api/media_contents/brand/{uuid}
            ←  MediaContentDto[]

useBrands       →  GET /api/brands
            ←  BrandDto[]
```

🚀 Quick Fix Summary

Minimal changes to make it work:

1. Change `/api/media_contents/brand/{brandId}` to accept `Guid`
2. Add `?brandId={guid}` query param to `/api/schedules`
3. Add CORS for `http://localhost:5173`
4. Return objects from POST/PUT (not void)

After these changes, restart your backend and the frontend should work!